TECHNICAL ASSIGNMENT SUBMISSION

Aarnex Tech Backend Developer

Complete Production-Grade Implementation Using MVC + TypeORM

100% TESTED & VERIFIED • 16/16 TESTS PASSED

Architecture: Strict MVC (Models, Controllers, Services, Repositories, Routes)

Stack: Node.js, Express.js, TypeScript, PostgreSQL (Neon / Docker), TypeORM, Zod, bcrypt, JWT, Pino

Database Mode: `synchronize: false` with versioned migrations & deterministic seed

Key Features: Pessimistic Locking (SELECT ... FOR UPDATE), Integer Paise Money Representation, JWT Rotation with Replay Invalidation

Interactive Swagger UI: `http://localhost:5000/api-docs`

---

1. Architecture Overview & Technology Stack

This project is a multi-branch retail & inventory management enterprise backend service engineered to withstand high-concurrency race conditions, prevent financial floating-point rounding errors, enforce branch-level data isolation, and guarantee absolute auditability.

Core Architectural Highlights

- **Strict MVC Layering**: Separation of HTTP handling (Controllers), Business transactions & validation (Services), Database queries (Repositories), and Entities (Models).
- **PostgreSQL Engine Constraints**: Native `CHECK` constraints (`quantity >= 0` , `price_paise >= 0`) and compound unique keys (`(branch_id, product_id)` , `(branch_id, email)`) ensure integrity at the database engine level.
- **Concurrency Control**: `SELECT ... FOR UPDATE` pessimistic row-level locking with deterministic lexicographical product sorting eliminates deadlocks and overselling.
- **Monetary Precision**: All financial calculations are stored and calculated in integer paise (1 INR = 100 paise). Floating-point math is completely eliminated.
- **Defense-in-Depth Authentication**: Access tokens (15m expiration) + Refresh tokens stored as SHA-256 hashes with atomic single-use rotation. Replay attacks trigger immediate family-wide session invalidation.

Layer / Function	Selected Technology	Design Rationale
Runtime & Language	Node.js (v20+) + TypeScript 5.4	Strict end-to-end type safety, modern async/await, and predictable execution.
Web Framework	Express.js 4.19	Lightweight, battle-tested HTTP routing with modular middleware composition.
Database & ORM	PostgreSQL 16 + TypeORM 0.3.20	Enterprise relational features, QueryRunner transaction controls, and native locking. (No Prisma).
Request Validation	Zod 3.23	Declarative runtime schema validation with strong TypeScript inference.
Security & Headers	Helmet 7.1 + express-rate-limit 7.2	OWASP security headers and brute-force mitigation on authentication endpoints.
Cryptography	bcrypt 5.1 + crypto (SHA-256)	Adaptive salted password hashing and secure token fingerprint storage.
Documentation	OpenAPI 3.0.3 + Swagger UI Express	Live interactive documentation hosted at `/api-docs` outside the API prefix.
Testing	Jest 29 + Supertest 7 + ts-jest	End-to-end integration tests directly against PostgreSQL database instances.

2. Database Schema (10 TypeORM Entities)

Entity / Table	Key Columns & Data Types	Engine Constraints & Indexes	Purpose
branches	`id (UUID)`, `name`, `code`, `address`, `phone`, `is_active`	PK `id`, Unique(`name`), Unique(`code`)	Physical branches (Mumbai Central, Bangalore Tech Park).
users	`id (UUID)`, `email`, `password_hash`, `full_name`, `role`, `branch_id`	PK `id`, Unique(`email`), FK(`branch_id`)	Roles: `SUPER_ADMIN`, `BRANCH_MANAGER`, `STAFF`.
products	`id (UUID)`, `sku`, `name`, `description`, `price_paise (INT)`	PK `id`, Unique(`sku`), `CHECK(price_paise >= 0)`	Catalog items. Prices stored strictly in integer paise.
customers	`id (UUID)`, `branch_id (UUID)`, `name`, `email`, `phone`	PK `id`, Unique(`branch_id, email`), FK(`branch_id`)	Branch-scoped customer profiles enforcing branch isolation.
inventories	`id (UUID)`, `branch_id`, `product_id`, `quantity (INT)`	PK `id`, Unique(`branch_id, product_id`), `CHECK(quantity >= 0)`	Stock per branch. Quantity cannot become negative.
stock_movements	`id (UUID)`, `branch_id`, `product_id`, `quantity_change`, `balance_after`, `reason`	PK `id`, FK(`branch_id`), FK(`product_id`), FK(`actor_id`)	Immutable audit ledger of every inventory change.
orders	`id (UUID)`, `order_number`, `branch_id`, `customer_id`, `status`, `total_amount_paise`	PK `id`, Unique(`order_number`), `CHECK(total_amount_paise >= 0)`	Status: `PLACED`, `PACKED`, `DISPATCHED`, `DELIVERED`, `CANCELLED`.
order_items	`id (UUID)`, `order_id`, `product_id`, `quantity`, `unit_price_paise`, `subtotal_paise`	PK `id`, FK(`order_id` CASCADE), `CHECK(quantity > 0)`	Historical line item snapshot preserving price at purchase time.
refresh_tokens	`id (UUID)`, `user_id`, `token_hash`, `is_revoked`, `replaced_by_token_id`	PK `id`, FK(`user_id` CASCADE), Index(`token_hash`)	Hashed refresh token store supporting single-use rotation.
audit_logs	`id (UUID)`, `actor_id`, `actor_email`, `action`, `entity_type`, `metadata (JSONB)`	PK `id`, Index(`action`), Index(`entity_type, entity_id`)	System-wide activity ledger with automatic secret redaction.

3. Complete API Catalog (All 22 Endpoints)

Method	Endpoint	Authorization	Description
POST	`/api/v1/auth/login`	Public (Rate-limited)	Login with email & password. Returns access & refresh tokens.
POST	`/api/v1/auth/refresh`	Public	Rotate tokens with `{ refreshToken }`. Replay protected.
POST	`/api/v1/auth/logout`	Authenticated	Invalidate active refresh token session.
GET	`/api/v1/auth/me`	Authenticated	Retrieve current user profile and assigned branch.
GET	`/api/v1/branches`	Authenticated	List all active branches in the company.
POST	`/api/v1/branches`	`SUPER_ADMIN`	Create branch with name, code, address, phone.
POST	`/api/v1/users`	`SUPER_ADMIN`	Create user with role and branch assignment.
PATCH	`/api/v1/users/:id/status`	`SUPER_ADMIN`	Activate or deactivate user (instantly revokes sessions).
GET	`/api/v1/products`	Authenticated	Search & paginate products: `?search=...&page=1&limit=20`.
GET	`/api/v1/products/:id`	Authenticated	Get single product details and price in paise.
POST	`/api/v1/products`	`SUPER_ADMIN`	Create product with unique SKU and price in paise.
PATCH	`/api/v1/products/:id`	`SUPER_ADMIN`	Update product name, description, price, or status.
GET	`/api/v1/customers`	Branch-Isolated	List customers belonging to caller's assigned branch.
POST	`/api/v1/customers`	Branch-Isolated	Create customer scoped to caller's branch.
GET	`/api/v1/branches/:id/inventory`	Authorized Branch	Retrieve stock levels for all products in branch.
POST	`/api/v1/branches/:id/inventory/adjustments`	`SUPER_ADMIN`, `BRANCH_MANAGER`	Adjust stock atomically with quantity_change (+/-). Staff forbidden.
GET	`/api/v1/orders`	Authorized Branch	List branch orders with pagination and status filters.
GET	`/api/v1/orders/:id`	Authorized Branch	Get order details with line items and historical prices.
POST	`/api/v1/orders`	Authorized Branch	Place order atomically under pessimistic row lock.
PATCH	`/api/v1/orders/:id/status`	Authorized Branch	Advance status: `PLACED` → `PACKED` → `DISPATCHED` → `DELIVERED`.
POST	`/api/v1/orders/:id/cancel`	Authorized Branch	Cancel order & restore stock. Rejects dispatched orders.

Method	Endpoint	Authorization	Description
GET	`/api/v1/reports/branches/:id/summary`	`SUPER_ADMIN`, `BRANCH_MANAGER`	Date-range financial report (`from=YYYY-MM-DD&to=YYYY-MM-DD`).
GET	`/api/v1/audit-logs`	`SUPER_ADMIN`, `BRANCH_MANAGER`	Query audit trail with automatic credential redaction.
GET	`/api/v1/health`	Public	Service health check executing active `SELECT 1` ping.

4. Pre-Seeded Testing Credentials & Entity UUIDs

Role	Email	Password	Assigned Branch	Permissions
SUPER_ADMIN	`admin@aarnex.com`	`Admin@123456`	Global	All branches, user admin, full catalog & inventory control.
BRANCH_MANAGER	`manager.mum@aarnex.com`	`User@123456`	Mumbai Central	Own-branch orders, inventory adjustments, and reports.
STAFF	`staff.mum@aarnex.com`	`User@123456`	Mumbai Central	Own-branch customers and orders. (Cannot adjust stock).
BRANCH_MANAGER	`manager.blr@aarnex.com`	`User@123456`	Bangalore Tech Park	Own-branch orders, inventory adjustments, and reports.
STAFF	`staff.blr@aarnex.com`	`User@123456`	Bangalore Tech Park	Own-branch customers and orders.

Live PostgreSQL Database UUIDs for Testing

- **Mumbai Central Branch ID**: `7dcdea18-efda-4456-aeb5-3feeaa9ee2c9`
- **Bangalore Tech Park Branch ID**: `047d7ab5-ac1b-4ce9-9fb7-dcdc72220ede`
- **Mechanical Keyboard RGB Product ID**: `18e1e15c-8488-4e32-9ef5-f173f3d2b6c7` (₹4,999.00 = `499900` paise)
- **Ergonomic Mouse Product ID**: `fedbc453-d7e3-4db9-a5c5-430f447f9f57` (₹2,499.00 = `249900` paise)
- **Mumbai Customer ID (Aarav Sharma)**: `12e9af09-2d44-4819-ae0d-0433780b9f4f`
- **Bangalore Customer ID (Rohan Iyer)**: `c5fdbd20-64b0-4263-bfdf-ae63be5bdb8f`

5. Acceptance Test Results (100% Verified)

#	Acceptance Test Scenario	Status	Tested Invariant
1	Protected endpoint without token returns 401	PASSED	JWT middleware blocks unauthenticated requests.
2	Branch A staff cannot access Branch B data	PASSED	Cross-branch access returns 403 Forbidden.
3	Incorrect password rejected; sensitive fields absent	PASSED	Bcrypt comparison fails on bad password; `password_hash` omitted.
4	Database price overrides client price	PASSED	Client submitted 100 paise; order created with DB price 499900 paise.
5	Insufficient stock causes transaction rollback	PASSED	Order exceeding stock rolls back completely (409 Conflict).
6	Two concurrent orders for last unit result in 1 order	PASSED	Pessimistic row lock guarantees 1 order commits, 1 gets 409. Final stock: 0.
7	Cancellation restores stock exactly once under concurrency	PASSED	Order lock prevents duplicate restore; stock incremented exactly once.
8	Invalid order status transitions are rejected	PASSED	State machine rejects illegal jumps and cancels after dispatch.
9	Branch manager cannot adjust another branch's stock	PASSED	Cross-branch stock adjustment returns 403 Forbidden.
10	Date-range reports match persisted orders	PASSED	SQL aggregate calculations match persisted DB records across UTC bounds.

6. Engineering Trade-offs Summary

- **Pessimistic Locking vs Optimistic Locking**: In inventory reservation, optimistic concurrency results in high abort/retry rates during flash sales. Pessimistic `SELECT ... FOR UPDATE` serializes access per SKU, eliminating wasted work.
- **Deterministic Lock Ordering**: Multi-item orders sort product IDs lexicographically before locking, mathematically eliminating lock-inversion deadlocks.
- **Integer Paise Currency**: IEEE 754 floating-point errors are eliminated by storing all currency as 32-bit integers in minor currency units (paise).
